# Práctica de estrategias algorítmicas: Ramificación y poda

Alejandro García García, Marcelo Fort Muñoz

## Contenido

1-Introducción ..... 1

2-Resultados de la experimentación ..... 1

3-Cambios para el caso de minimización..... 2

## 1-Introducción

En esta práctica nos centramos en la implementación y análisis de la estrategia de ramificación y poda aplicada a un problema de asignación, en el que buscamos la combinación de asignaciones entre unas n tareas y n personas de forma que se maximice el beneficio.

Esta técnica procura explorar el espacio de soluciones de manera más "inteligente" que otros algoritmos como puede ser el de vuelta atrás. Así, en la exploración de soluciones, se descartan aquellas que no conducen a soluciones finales óptimas.

El problema propuesto lo afrontamos desde dos enfoques: uno de estimación trivial y otro de estimación precisa. La diferencia entre estos dos tipos de estimación radica en las estrategias que uno y otro emplean para calcular las cotas inferiores y superiores dentro del algoritmo; mientras que el caso de estimaciones triviales emplearemos estrategias más simples para el cálculo de las cotas, cuando usemos estimaciones precisas, utilizaremos funciones más avanzadas, como la de asignación voraz en el caso de la cota inferior, para realizar sus operaciones.

El método elegido para la selección de nodos en nuestro trabajo es la estrategia MB-LIFO, la cual mantiene una preferencia a la hora de explorar de manera más exhaustiva aquellas ramas que parezcan más prometedoras en términos de beneficio estimado.

También debemos tener en cuenta que, en el contexto de nuestra práctica, utilizamos matrices de dos tamaños distintos (N=3 y N=6). Este factor tiene un impacto significativo en el rendimiento y la complejidad computacional del algoritmo.

## 2-Resultados de la experimentación

Para la experimentación probamos con dos matrices distintas, una primera de tamaño 3 y otra de tamaño 6. Comparando los resultados obtenidos tras la ejecución de nuestro código observamos como el tamaño influye de forma en que, al aumentar, también aumenta el número de nodos, siendo este incremento muchísimo mayor en el caso de estimación trivial.

Esto puede deberse a que la estrategia precisa es más eficiente, al emplear técnicas de cálculo de cotas y selección de nodos más sofisticadas, esto es, a mayor complejidad de cálculo, más se reducen los nodos explorados. De esta manera, con los datos obtenidos, podemos afirmar que la estrategia para una

ramificación y poda más eficientes para nuestro problema es la que emplea las funciones de estimación precisa.

| N | Nº. de nodos explorados (trivial) | Nº. de nodos explorados (precisa) |
|---|---|---|
| 3 | 9 | 4 |
| 6 | 1248 | 109 |

*Tabla 1:Resultados de ramificación y poda (Alejandro)*

| N | Nº. de nodos explorados (trivial) | Nº. de nodos explorados (precisa) |
|---|---|---|
| 3 | 6 | 4 |
| 6 | 1248 | 106 |

*Tabla 2:Resultados de ramificación y poda (Marcelo)*

Comparando la estrategia de ramificación y poda con otras estrategias, como la de "backtracking", también podemos concluir que esta es más eficiente en lo que a número de nodos explorados se refiere, obteniendo valores mucho menores con la técnica que protagoniza este informe. Esta mejora en los resultados se podría deber a que en ramificación y poda no se continúan evaluando aquellas soluciones que no conducirán a una solución final óptima, mientras que en "backtracking" sí.

## 3-Cambios para el caso de minimización

Para minimizar el resultado modificando el algoritmo de ramificación y poda existen al menos 2 caminos posibles a tomar. El primero consiste en multiplicar la matriz de entrada por -1 y, haciendo unas pequeñas modificaciones (presentes en el código), este da la solución optimizada a mínimos. La otra forma es mantener la matriz original, pero, alterar bastantes más elementos de la lógica interna que constituye el algoritmo.

Las modificaciones más notables que ha sufrido el código para realizar este segundo método (en el cual no se cambia el signo de los elementos) son:

I. Al inicializar el vector "solución" su "beneficio actual" pasa de ser 0 a un número muy grande (en nuestro caso "INT_MAX"). Esto nos asegura que cualquier primera solución (por muy grande que sea) pueda ser aceptada.
II. Se ha cambiado la estrategia de selección de MB-LIFO a LC-LIFO, es decir: Se elige entre los nodos presentes en la lista de nodos vivos aquel que tenga menor coste estimado y, en caso de empate, el que entrase más tarde entre los empatados.
III. Es de notar que en el caso de la estimación trivial no ha sido necesaria una modificación del método de cálculo de las cotas inferior y superior; en el caso de la estimación precisa, la "asignación voraz" ha pasado de usarse para el cálculo de la cota inferior a usarse para el cálculo de la cota superior, y la nueva cota inferior se basa en suponer que podremos asignar la tarea de menor coste a cada usuario sin importar si ha sido asignada previamente o no.
IV. La variable "Cota" ha pasado a almacenar (para las soluciones parciales) los menores valores de las cotas superiores que vayamos encontrando y (si son soluciones finales) las menores sumas totales hasta ese momento.
V. La pode de nodos vivos pasa a realizarse sobre aquellos cuya cota inferior sea mayor o igual que la "Cota" que tengamos en cada momento.